Panduan Membuat Soal Competitive Programming

Aturan dan langkah menulis soal, test case, batas waktu, dan poin untuk kontes CP di website 14th NPLC, supaya setiap soal bisa dinilai otomatis oleh Judge0 dengan adil.

1 Ringkasan aturan

2 Cara penilaian bekerja

3 Mengisi form soal

4 Menulis deskripsi soal

5 Membuat test case

6 Menentukan batas waktu dan memori

7 Poin, tingkat kesulitan, dan skor

8 Publikasi dan perubahan soal

9 Contoh soal lengkap

10 Checklist sebelum publikasi

Untuk
Panitia penulis soal

Tempat input soal
`/admin/cp → Soal baru`

Versi
16 Oktober 2026

1. Ringkasan aturan

Kalau hanya sempat membaca satu halaman, baca bagian ini. Setiap aturan dijelaskan lagi di bagian berikutnya.

1 **Satu input, satu jawaban benar.**
Output dibandingkan persis dengan jawaban. Soal dengan banyak jawaban yang sama-sama benar (misalnya "cetak salah satu solusi") tidak bisa dinilai.

2 **Format input dan output harus jelas dan lengkap.**
Tulis apa saja yang ada di setiap baris, dipisah spasi atau baris baru, dan bagaimana output dicetak. Peserta tidak boleh menebak.

3 **Batasan (constraints) wajib ditulis dan wajib diuji.**
Contoh: `1 ≤ n ≤ 10^5`. Minimal satu test tersembunyi memakai ukuran maksimum, supaya solusi yang terlalu lambat gagal.

4 **Jawaban test dibuat dari solusi resmi yang sudah diuji, bukan ditulis tangan.**
Satu jawaban salah di test tersembunyi membuat semua tim kehilangan poin secara tidak adil.

5 **Minimal 2 test contoh dan 4 test tersembunyi.**
Contoh terlihat peserta dan dipakai tombol Jalankan. Tersembunyi hanya untuk menilai.

6 **Batas waktu ditentukan dari solusi Python.**
Satu batas waktu berlaku untuk Python, C++, dan Java. Python paling lambat, jadi solusi resmi dalam Python harus lolos dengan aman.

7 **Selesaikan soal dan test case sebelum kontes dimulai.**
Mengubah test case saat kontes berlangsung menghapus hasil per-test dari semua kiriman lama untuk soal itu.

8 **Tidak ada soal interaktif, tidak ada file, tidak ada internet.**
Program hanya membaca dari input standar (stdin) dan menulis ke output standar (stdout).

2. Cara penilaian bekerja

Saat peserta menekan **Kirim**, server menjalankan kode di Judge0 (server penilai milik panitia) untuk **setiap** test case, lalu membandingkan output program dengan jawaban yang kamu tulis. Jawaban tidak pernah dikirim ke Judge0 maupun ke browser peserta.

Bahasa yang tersedia

Bahasa	Versi	Catatan untuk penulis soal
Python	Python 3.8.1	Paling lambat. Fitur setelah 3.8 (misalnya `match`) tidak tersedia.
C++	GCC 9.2.0	Paling cepat. Ingatkan peserta soal overflow: hasil di atas 2^{31} butuh `long long`.
Java	OpenJDK 13.0.1	Class harus bernama `public class Main`, tanpa `package`.

Bagaimana output dibandingkan

Sebelum dibandingkan, output program dan jawaban dirapikan dengan aturan ini:

- Akhir baris Windows (`\r\n`) diubah jadi `\n`.
- Spasi di **akhir** setiap baris diabaikan.
- Baris kosong di **akhir** output diabaikan.

Selain itu semuanya harus sama persis: spasi di tengah baris, huruf besar/kecil, jumlah angka di belakang koma, dan urutan baris.

Akibatnya untuk soal

- Kalau outputnya bilangan desimal, **tentukan presisinya**, misalnya "cetak tepat 2 angka di belakang koma". Tidak ada toleransi error.
- Kalau output berisi teks, tulis persis dengan huruf besar/kecilnya, misalnya `YES` / `NO`.
- Kalau output berisi daftar, tentukan urutan dan pemisahnya, misalnya "dipisah satu spasi, urut naik".

Hasil penilaian (verdict)

Hasil sebuah kiriman ditentukan oleh **test pertama yang gagal**. Kalau semua lolos, hasilnya Diterima.

Hasil	Artinya
Diterima	Semua test lolos dalam batas waktu dan memori.
Jawaban salah	Program selesai, tapi outputnya berbeda.
Melebihi batas waktu	Waktu CPU melewati batas waktu soal.
Error kompilasi	Kode tidak bisa dikompilasi; tidak ada test yang dijalankan.
Runtime error	Program crash: indeks di luar batas, pembagian dengan nol, memori habis, exception, dan sejenisnya.
Penilaian gagal di server	Judge0 bermasalah. Bukan salah peserta. Kiriman tetap tercatat, tapi tidak bisa menurunkan skor karena yang dihitung hanya skor terbaik.

Batas lain dari Judge0: output program maksimal 1 MB; waktu nyata (wall-clock) maksimal 3 × batas waktu + 1 detik, paling lama 20 detik.

3. Mengisi form soal

Buka `/admin/cp`, lalu tekan **Soal baru** di kartu Soal. Semua field berikut ada di form itu.

Field	Aturan	Petunjuk
Level	Bilangan bulat 1–999, unik dalam satu lomba	Urutan soal di daftar peserta. Mulai dari 1, urutkan dari yang termudah.
Judul	Wajib, maksimal 120 karakter	Singkat dan menggambarkan soal. Jangan membocorkan solusinya (hindari "Binary Search Sederhana").
Tingkat kesulitan	Mudah, Sedang, atau Sulit	Ditampilkan sebagai label di daftar soal.
Poin	Bilangan bulat 1–100.000	Skor maksimum soal ini. Lihat bagian 7.
Deskripsi soal	Wajib, mendukung Markdown	Cerita dan tugas soal. Pakai tab Pratinjau untuk mengecek tampilannya.
Format input	Maksimal 5.000 karakter	Isi setiap baris input, berurutan.
Format output	Maksimal 5.000 karakter	Apa yang dicetak, dan bagaimana formatnya.
Batasan	Maksimal 5.000 karakter	Rentang setiap nilai, misalnya `1 ≤ n ≤ 10^5`, `-10^9 ≤ a_i ≤ 10^9`.
Batas waktu	100–15.000 ms (bawaan 1.000)	Waktu CPU per test. Lihat bagian 6.
Batas memori	16.384–512.000 KB (bawaan 262.144 = 256 MB)	Biarkan 256 MB kecuali soal memang menguji pemakaian memori.
Test case	1–100 test; input dan jawaban masing-masing maksimal 2 MB	Centang "Contoh" untuk test yang boleh dilihat peserta. Lihat bagian 5.
Publikasikan	Perlu minimal 1 test case	Hanya soal yang dipublikasikan yang terlihat peserta, dan hanya saat kontes berlangsung.

Menulis dengan Markdown

Deskripsi, format input/output, dan batasan ditampilkan sebagai Markdown. Yang didukung: paragraf, **tebal**, *miring*, daftar, tabel, `` `kode` ``, blok kode, dan tautan. HTML tidak ditampilkan.

Rumus LaTeX tidak didukung

Tulisan seperti `$10^5$` atau `\le` akan tampil apa adanya. Tulis rumus sebagai teks: `10^5`, `a_i`, dan simbol Unicode seperti `≤ ≥ × ≠`. Bungkus dengan `` `backtick` `` supaya tampil sebagai kode.

4. Menulis deskripsi soal

Deskripsi yang baik bisa dipahami peserta SMA tanpa bertanya ke panitia. Saat kontes, peserta tidak bisa bertanya.

a **Mulai dari tugasnya, baru ceritanya.**
Cerita boleh ada, tapi kalimat yang menjelaskan apa yang harus dihitung harus mudah ditemukan.

b **Definisikan setiap istilah.**
"Subarray", "palindrom", "berurutan": jelaskan, atau beri contoh kecil di deskripsi.

c **Jawaban harus tunggal.**
Kalau ada beberapa jawaban yang mungkin, tentukan aturan pemilihnya, misalnya "jika ada beberapa, cetak yang terkecil".

d **Kasus khusus ditulis eksplisit.**
Misalnya: "Jika tidak ada jawaban, cetak `-1`."

e **Sertakan penjelasan untuk contoh.**
Satu-dua kalimat yang menjelaskan kenapa contoh 1 menghasilkan output itu sangat membantu peserta.

f **Gunakan satu bahasa secara konsisten.**
Pilih Bahasa Indonesia atau Inggris untuk semua soal, dan jangan dicampur dalam satu soal.

5. Membuat test case

Test case menentukan kualitas soal. Test yang lemah membuat solusi salah ikut diterima; test yang salah membuat solusi benar ditolak.

Dua jenis test

Jenis	Siapa yang melihat	Dipakai untuk	Jumlah
Contoh	Peserta (input dan jawaban)	Tombol **Jalankan**, dan ikut dinilai saat Kirim	2–3
Tersembunyi	Hanya panitia	Tombol **Kirim**	4–20

Skor dihitung dari *jumlah test yang lolos dibagi jumlah semua test*, termasuk test contoh. Semakin banyak test, semakin halus pembagian skor parsialnya.

Test tersembunyi yang wajib ada

- **Nilai terkecil:** `n = 1`, nilai 0, string kosong jika diizinkan batasan.
- **Nilai terbesar:** `n` maksimum dan nilai maksimum. Ini yang menangkap overflow dan solusi lambat.
- **Kasus khusus:** semua nilai sama, nilai negatif, sudah terurut, terurut terbalik, jawaban `-1`.
- **Kasus acak** berukuran sedang, untuk menguji kebenaran secara umum.

Cara membuat jawaban test

1. Tulis **solusi resmi** (sebaiknya dalam C++ dan Python), lalu uji dengan semua contoh.
2. Buat input besar dengan program generator, bukan diketik manual.
3. Jalankan solusi resmi untuk menghasilkan jawaban setiap test.
4. Kalau sempat, minta panitia lain menulis solusinya sendiri dan bandingkan jawabannya (cross-check).
5. Tempel input dan jawaban ke form. Jawaban harus diakhiri dengan baris baru atau tidak, keduanya diterima.

```
# Contoh: membuat jawaban dari solusi resmi
python3 gen.py 7 > test7.in          # generator dengan seed 7
python3 solusi.py < test7.in > test7.out
# lalu tempel isi test7.in ke "Input" dan test7.out ke "Output yang diharapkan"
```

Batas ukuran test

Input dan jawaban masing-masing maksimal 2 MB per test, dengan maksimal 100 test per soal. Kalau batasan soal butuh input lebih besar dari 2 MB, kecilkan batasannya.

6. Menentukan batas waktu dan memori

Judge0 tidak bisa mengukur kompleksitas algoritma secara langsung. Yang membedakan solusi cepat dan lambat adalah **test besar + batas waktu**: solusi dengan kompleksitas yang salah akan Melebihi batas waktu di test besar.

1. Tulis solusi resmi dengan kompleksitas yang diharapkan, dalam **Python**.
2. Jalankan di test terbesar **di komputer sendiri** dan catat waktunya. Tombol Jalankan di website hanya memakai test contoh, jadi tidak bisa dipakai untuk mengukur test tersembunyi.
3. Atur batas waktu sekitar **2–3 kali** waktu solusi Python itu, dibulatkan ke 500 ms terdekat.

4. Pastikan solusi naif (misalnya $O(n^2)$ untuk $n = 10^5$) tetap melebihi batas waktu di C++. Kalau tidak, besarkan batasan `n`.

Ukuran n	Kompleksitas yang lolos	Saran batas waktu
$n \le 1.000$	$O(n^2)$	1.000 ms
$n \le 10^5$	$O(n \log n)$	1.000–2.000 ms
$n \le 10^6$	$O(n)$	2.000–3.000 ms

Angka di tabel adalah titik awal untuk soal tingkat SMA. Tetap ukur dengan solusi Python sendiri.

7. Poin, tingkat kesulitan, dan skor

Rumus skor

```
efisiensi = antara 0,7 dan 1,0, makin dekat batas waktu/memori makin kecil
skor      = poin × (test lolos ÷ total test) × efisiensi
```

- Efisiensi dihitung dari test terlambat (bobot 70%) dan pemakaian memori tertinggi (bobot 30%) dibanding batas soal. Solusi yang jauh di bawah batas mendapat 1,0; yang hampir menyentuh batas mendapat 0,7.
- Kiriman tanpa test yang lolos mendapat 0.
- Yang dihitung hanya **skor terbaik** tiap tim per soal. Kiriman yang lebih buruk tidak menurunkan skor.
- Peringkat diurutkan dari total poin, lalu siapa yang **lebih dulu** mencapai poin itu.

Saran poin

Tingkat kesulitan	Ciri soal	Saran poin
Mudah	Satu konsep, implementasi langsung (perulangan, kondisi, array)	100
Sedang	Butuh algoritma standar (sorting, prefix sum, greedy, binary search)	200–300
Sulit	Gabungan konsep atau algoritma lanjut (DP, graf)	400–500

Ini saran, bukan aturan sistem. Yang penting: soal yang lebih sulit diberi poin lebih besar, dan skala poin konsisten di semua soal satu lomba.

8. Publikasi dan perubahan soal

- Soal bisa disimpan sebagai **Draf** dulu. Peserta tidak pernah melihat draf.
- Soal hanya bisa dipublikasikan kalau sudah punya minimal satu test case.
- Soal yang dipublikasikan terlihat peserta **hanya saat kontes berlangsung**. Sebelum kontes dimulai dan saat dijeda, soal tersembunyi.
- Menyimpan soal **mengganti semua test case** soal itu. Hasil per-test dari kiriman lama ikut terhapus, tapi skor yang sudah didapat tetap.

Jangan mengubah test case saat kontes berjalan

Kalau ternyata ada test yang salah saat kontes, jeda kontes dulu, diskusikan dengan panitia, perbaiki test, umumkan ke semua tim, lalu lanjutkan. Kiriman lama tidak dinilai ulang otomatis.

9. Contoh soal lengkap

Soal Level 1 yang sudah ada di website, ditulis sesuai aturan di panduan ini.

Level 1 · Mudah · 100 poin · Batas waktu 1.000 ms · Memori 256 MB

Judul

Sum of Two Numbers

Deskripsi soal

Given two integers `a` and `b`, print their sum.

Format input

One line with two integers `a` and `b`, separated by a space.

Format output

Print `a + b`.

Batasan

`-10^9 ≤ a, b ≤ 10^9`

Test	Jenis	Input	Jawaban	Tujuan
1	Contoh	`3 5`	`8`	Kasus biasa
2	Contoh	`-4 10`	`6`	Bilangan negatif
3	Tersembunyi	`0 0`	`0`	Nilai nol
4	Tersembunyi	`1000000000 1000000000`	`2000000000`	Overflow `int` 32-bit
5	Tersembunyi	`-1000000000 -999999999`	`-1999999999`	Batas bawah
6	Tersembunyi	`123456 -654321`	`-530865`	Hasil negatif

Perhatikan test 4: solusi C++ yang memakai `int` akan mendapat Jawaban salah karena 2.000.000.000 melebihi batas `int`. Batasan `10^9` di soal memberi tahu peserta bahwa ini mungkin terjadi.

10. Checklist sebelum publikasi

- [ ] Level unik dan urut dari yang termudah
- [ ] Judul tidak membocorkan solusi
- [ ] Tugas soal jelas dan jawabannya tunggal
- [ ] Kasus khusus (tidak ada jawaban, seri) dijelaskan
- [ ] Format input menyebut isi setiap baris
- [ ] Format output menyebut presisi, huruf, dan pemisah
- [ ] Batasan ditulis untuk setiap nilai
- [ ] Rumus ditulis tanpa LaTeX, dicek di Pratinjau
- [ ] Ada 2–3 test contoh dengan penjelasan
- [ ] Ada test nilai terkecil dan terbesar
- [ ] Ada test kasus khusus dan test acak
- [ ] Semua jawaban dibuat dari solusi resmi
- [ ] Jawaban di-cross-check oleh panitia lain
- [ ] Solusi resmi Python lolos dengan batas waktu
- [ ] Solusi naif gagal di test terbesar
- [ ] Setiap test di bawah 2 MB
- [ ] Poin sesuai tingkat kesulitan
- [ ] Soal dicoba lewat tombol Jalankan sebagai admin